```c
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n];
    float total_tat = 0, total_wt = 0;

    // Input Section
    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
    }

    // Completion Time Calculation
    // First process special case
    ct[0] = at[0] + bt[0];

    // Subsequent processes
    for(i = 1; i < n; i++) {
        // If the CPU is idle (current AT > previous process CT)
        if(at[i] > ct[i-1])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }

    // TAT and WT Calculation
    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        total_tat += tat[i];
        total_wt += wt[i];
    }

    // Output Table
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

    return 0;
}
```

```
Enter number of processes: 4

Process 1
Arrival Time: 0
Burst Time: 2

Process 2
Arrival Time: 1
Burst Time: 2

Process 3
Arrival Time: 5
Burst Time: 3

Process 4
Arrival Time: 6
Burst Time: 4

Process AT      BT      CT      TAT     WT
P1      0       2       2       2       0
P2      1       2       4       3       1
P3      5       3       8       3       0
P4      6       4       12      6       2

Average Turnaround Time = 3.50
Average Waiting Time = 0.75

Process returned 0 (0x0)   execution time : 40.760 s
Press any key to continue.
```

```c
#include <stdio.h>

int main() {
    int n, i, time = 0, count = 0, min, index;

    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1) return 1;

    int at[n], bt[n], ct[n], tat[n], wt[n], done[n];

    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        done[i] = 0; // Initialize all processes as not done
    }

    while(count < n) {
        min = 9999;
        index = -1;

        for(i = 0; i < n; i++) {
            // Check if process has arrived and is not yet finished
            if(at[i] <= time && done[i] == 0) {
                if(bt[i] < min) {
                    min = bt[i];
                    index = i;
                }
            }
        }

        if(index == -1) {
```

```
            // No process has arrived yet, just move time forward
            time++;
        } else {
            // Process the selected shortest job
            ct[index] = time + bt[index];
            time = ct[index];
            tat[index] = ct[index] - at[index];
            wt[index] = tat[index] - bt[index];
            done[index] = 1;
            count++;
        }
    }

    float total_tat = 0, total_wt = 0;

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
        total_tat += tat[i];
        total_wt += wt[i];
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

    return 0;
}
```

```
Enter number of processes: 4

Process 1
Arrival Time: 0
Burst Time: 6

Process 2
Arrival Time: 0
Burst Time: 8

Process 3
Arrival Time: 0
Burst Time: 7

Process 4
Arrival Time: 0
Burst Time: 3

Process AT      BT      CT      TAT     WT
P1      0       6       9       9       3
P2      0       8       24      24      16
P3      0       7       16      16      9
P4      0       3       3       3       0

Average Turnaround Time = 13.00
Average Waiting Time = 7.00

Process returned 0 (0x0)   execution time : 122.804 s
Press any key to continue.
```

```c
#include <stdio.h>

int main() {
    int n, i, time = 0, completed = 0, min, index;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];

    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        rt[i] = bt[i]; // Initialize remaining time with burst time
    }

    // Logic for Shortest Remaining Time First
    while(completed < n) {
        min = 9999;
        index = -1;

        for(i = 0; i < n; i++) {
            // Check if process has arrived and isn't finished
            if(at[i] <= time && rt[i] > 0) {
                if(rt[i] < min) {
                    min = rt[i];
                    index = i;
                }
            }
        }
```

```
        if(index == -1) {
            // No process is ready, just move time forward
            time++;
        } else {
            rt[index]--;
            time++;

            // If process is finished
            if(rt[index] == 0) {
                ct[index] = time;
                tat[index] = ct[index] - at[index];
                wt[index] = tat[index] - bt[index];
                completed++;
            }
        }
    }

    float total_tat = 0, total_wt = 0;

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
        total_tat += tat[i];
        total_wt += wt[i];
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

    return 0;
}
```

```
Process 2
Arrival Time: 1
Burst Time: 5

Process 3
Arrival Time: 4
Burst Time: 1

Process 4
Arrival Time: 0
Burst Time: 6

Process 5
Arrival Time: 2
Burst Time: 3

Process AT      BT      CT      TAT     WT
P1      2       1       3       1       0
P2      1       5       11      10      5
P3      4       1       5       1       0
P4      0       6       16      16      10
P5      2       3       7       5       2

Average Turnaround Time = 6.60
Average Waiting Time = 3.40

Process returned 0 (0x0)   execution time : 22.443 s
Press any key to continue.
```